IMPLEMENTASI METODE *FASTER* R-CNN UNTUK DETEKSI TEKS DALAM *SCENE IMAGES*

Disusun Oleh:

M Aris Susanto	220491100002
Abduh Syauqi Ramadhan	220491100016
Fadzillah Akbar Subkhi	220491100042
Agus Murdiono	220491100045

PROGRAM STUDI TEKNIK MEKATRONIKA

FAKULTAS TEKNIK

UNIVERSITAS TRUNOJOYO MADURA

2025

ABSTRAK

Deteksi teks dalam gambar lingkungan nyata (*scene images*) merupakan salah satu tantangan utama dalam bidang pengolahan citra digital karena beragamnya bentuk, orientasi, ukuran, serta latar belakang visual yang kompleks. Penelitian ini bertujuan untuk mengimplementasikan metode *Region-based Convolutional Neural Network* (R-CNN), khususnya varian *Faster* R-CNN dengan *backbone* ResNet-50, untuk mendeteksi teks secara presisi dalam gambar nyata. Dataset yang digunakan adalah ICDAR 2015 yang telah dikonversi ke format COCO dan terdiri dari ribuan gambar beranotasi. Proses pelatihan model dilakukan menggunakan *framework* PyTorch selama lima epoch, dengan pendekatan transfer *learning* dari model yang sebelumnya telah dilatih pada dataset COCO. Evaluasi terhadap model dilakukan menggunakan metrik COCO dengan hasil *Mean Average Precision* (mAP) sebesar 29,8% pada rentang IoU 0,50 hingga 0,95, serta rata-rata *recall* sebesar 40%. Hasil ini menunjukkan bahwa metode *Faster* R-CNN memiliki potensi yang cukup baik dalam mendeteksi teks dalam gambar *scene*, walaupun masih terdapat ruang untuk peningkatan kinerja melalui penyesuaian parameter, teknik augmentasi data, dan pengoptimalan struktur *anchor box*.

Kata Kunci : *Faster* R-CNN, deteksi objek, ICDAR 2015, CNN, *deep learning*

BAB I
PENDAHULUAN

1.1. Latar Belakang

Di era digital saat ini, informasi visual seperti gambar dan video telah menjadi sumber data utama dalam berbagai aplikasi, termasuk sistem navigasi, pemantauan lalu lintas, penerjemahan otomatis, dan *augmented reality*. Salah satu komponen penting dari informasi visual adalah teks yang terdapat dalam gambar dikenal sebagai *scene text*. Deteksi dan ekstraksi teks dari gambar lingkungan nyata (*scene images*) merupakan tantangan dalam bidang *Computer Vision* karena teks dapat muncul dalam berbagai ukuran, orientasi, jenis *font*, pencahayaan, dan latar belakang yang kompleks.

Untuk mengatasi tantangan tersebut, pendekatan berbasis *Deep Learning* seperti *Region-based Convolutional Neural Networks* (R-CNN) telah menjadi metode populer dan efektif dalam mendeteksi objek termasuk teks dalam gambar. R-CNN dan varian lanjutannya seperti *Fast* R-CNN dan *Faster* R-CNN memungkinkan ekstraksi fitur visual dan lokalisasi objek secara presisi dalam satu arsitektur terpadu, menjadikannya pilihan yang relevan untuk *scene text detection*.

Meskipun begitu, implementasi metode *Faster* R-CNN dalam konteks deteksi teks masih relatif kurang dieksplorasi dibanding deteksi objek umum seperti kendaraan atau wajah. Maka dari itu, makalah ini bertujuan untuk mengevaluasi secara sistematis bagaimana metode *Faster* R-CNN dapat diimplementasikan dan disesuaikan untuk mendeteksi teks dalam *scene images* secara efisien.

1.2. Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana metode *Faster* R-CNN dapat digunakan untuk mendeteksi teks dalam gambar dengan latar kompleks (*scene images*)?
2. Seberapa akurat metode *Faster* R-CNN dalam mendeteksi dan mengekstraksi teks?

1.3. Tujuan Penelitian

Adapun tujuan dari dibuatnya proposal ini adalah ditunjukkan sebagai berikut:

1. Menerapkan metode *Faster* R-CNN dalam mendeteksi teks pada *scene images*.
2. Mengevaluasi performa metode *Faster* R-CNN dalam hal akurasi, presisi, dan kecepatan

BAB II
PENELITIAN TERKAIT

Pada tahun 2021, Muhammad Sulthon Yazid Basthomi, dan Fitri Utaminingrum melakukan penelitian dengan judul "Deteksi dan pengenalan plat nama ruangan menggunakan *faster-rcnn* dan *pytesseract* pada purwarupa kursi roda pintar." Penelitian ini berhasil mengembangkan sistem deteksi dan pengenalan plat nama ruangan pada kursi roda pintar menggunakan metode *Faster*-RCNN dan PyTesseract dengan *platform* Raspberry Pi 4B. Hasil pengujian menunjukkan bahwa sistem mampu mendeteksi plat nama ruangan dengan akurasi pendeteksian menggunakan Faster-RCNN 87% dan mengenali nama ruangan dengan rasio prediksi pengenalan menggunakan PyTesseract 77,73%. Hasil ini menunjukkan bahwa sistem memiliki potensi yang baik untuk mendukung navigasi otomatis di lingkungan rumah sakit [1].

Pada tahun 2023. Belinda Fridolin Darmanto, Casi Setianingsih, dan Randy Erfa Saputra, yang membahas Deteksi Pelanggaran pada Bahu Jalan Tol Menggunakan Algoritma Cascade R-CNN. Penelitian ini berhasil menerapkan algoritma Cascade R-CNN untuk mendeteksi pelanggaran di bahu jalan tol. Model yang dikembangkan mampu membedakan kendaraan yang melanggar dan tidak melanggar dengan akurasi tinggi, ditunjukkan oleh nilai AP 97,1% dan mAP@0.5:0.95 sebesar 74,8%. Hasil ini menunjukkan bahwa sistem dapat dikembangkan lebih lanjut untuk diintegrasikan ke dalam sistem pemantauan CCTV secara langsung [2].

Pada tahun yang sama, tahun 2023. Y. L. Chaitra , M. J. Roopa , M. T. Gopalakrishna ,M.D.Swetha, dan C. R. Aditya melakukan penelitian untuk mengembangkan metode *end-to-end* berbasis *deep learning* yang menggabungkan *Faster* R-CNN untuk deteksi lokasi teks dan EasyOCR untuk pengenalan teks dari gambar *scene*. Metode ini diuji pada dataset benchmark ICDAR13 dan ICDAR15, dan menunjukkan peningkatan akurasi deteksi (F-*score*) masing-masing sebesar 2.1% dan 1.4%, serta peningkatan akurasi pengenalan sebesar 1.8%. Hasil ini

membuktikan bahwa pendekatan yang diusulkan efektif untuk pengenalan teks dalam kondisi lingkungan kompleks dan beragam [3].

Pada satu tahun selanjutnya, tahun 2024. Nanang Cahyadi, Sherfina Salsabila, Sevierda Raniprima, dan Vivi Monita, melakukan penelitian berjudul " Tinjauan Literatur Teknik Pengenalan Plat Nomor Berbasis Deep Learning". Penelitian ini berhasil membahas pendekatan *deep learning* dalam sistem Automatic License Plate Recognition (ALPR), termasuk metode fusi, dua tahap, end-to-end, multi-cabang, dan generatif, serta juga membahas kelebihan. Hasil menunjukkan bahwa integrasi model deep learning modern seperti GAN, Transformer, dan Reinforcement Learning berpotensi meningkatkan akurasi, efisiensi, dan adaptivitas sistem ALPR di berbagai kondisi nyata [4].

Tabel 1. Penelitian Terkait

No	Judul	Penulis	Tahun	Kesimpulan
1.	Deteksi dan pengenalan plat nama ruangan menggunakan *faster-rcnn* Dan *pytesseract* pada purwarupa kursi roda pintar	Muhammad Sulthon Yazid Basthomi, dan Fitri Utaminingrum	2021	Penelitian ini berhasil mengembangkan sistem deteksi dan pengenalan plat nama ruangan menggunakan Faster-RCNN dan PyTesseract pada Raspberry Pi 4B. Hasilnya menunjukkan akurasi deteksi sebesar 87% dan pengenalan sebesar 77,73%, dengan integrasi perangkat keras mencapai 100%.
2.	Deteksi Pelanggaran pada Bahu Jalan Tol Menggunakan Algoritma	Belinda Fridolin Darmanto, Casi Setianingsih, dan Randy Erfa	2023	Penelitian ini berhasil menerapkan algoritma Cascade R-CNN untuk mendeteksi pelanggaran pada bahu jalan tol. Hasil

	Cascade R-CNN	Saputra		menunjukan model terbaik mencapai AP 97,1% dan mAP@0.5:0.95 sebesar 74,8%, menunjukkan bahwa dapat digunakan pada sistem pemantauan CCTV.
3.	Text Detection and Recognition from the Scene Images Using RCNN and EasyOCR	Y. L. Chaitra , M. J. Roopa , M. T. Gopalakrishna ,M.D.Swetha, dan C. R. Aditya	2023	Penelitian ini berhasil membahas deteksi dan pengenalan teks dalam *images* lingkungan nyata menggunakan Faster R-CNN dan EasyOCR. Hasil membuktikan bahwa Metode yang digunakan menunjukkan peningkatan F-score sebesar 2.1% pada ICDAR13 dan 1.4% pada ICDAR15, juga peningkatan pada akurasi pengenalan teks sebesar 1.8%.
4.	Tinjauan Literatur Teknik Pengenalan Plat Nomor Berbasis Deep Learning	Nanang Cahyadi, Sherfina Salsabila, Sevierda Raniprima, dan	2024	Artikel membahas keberhasilan terhadap berbagai pendekatan deep learning dalam sistem pengenalan plat nomor kendaraan (ALPR),

		Vivi Monita		seperti model dua tahap, end-to-end, dan multi-cabang.

BAB III
METODOLOGI PENELITIAN

Bab ini menjelaskan tahapan-tahapan yang dilakukan dalam implementasi metode *Faster Region-based Convolutional Neural Network* (R-CNN) untuk mendeteksi teks dalam *scene images*. Metode penelitian menggunakan pendekatan *deep learning* menggunakan data anotasi teks dari lingkungan nyata.

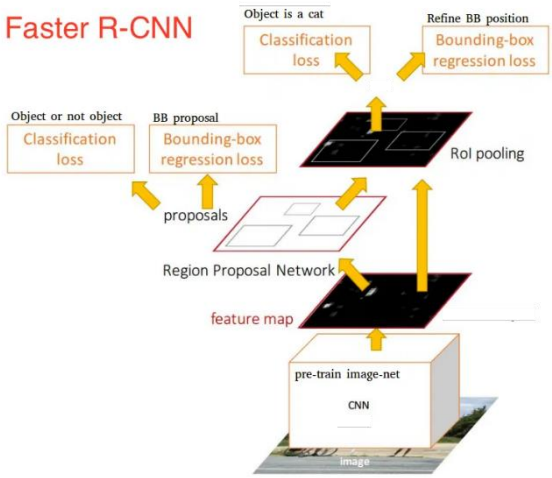
3.1. Dataset

Dataset yang digunakan dalam penelitian ini adalah ICDAR 2015 yang diperoleh melalui situs www.kaggle.com. Dataset ini dirancang khusus untuk mendukung tugas deteksi serta pengenalan teks pada gambar-gambar lingkungan nyata (*scene images*). Struktur dataset terdiri atas dua folder utama, yaitu folder *train* dan folder *test*. Masing-masing folder memiliki dua subfolder, yaitu *images* dan *gt*. Subfolder *images* berisi file citra dengan format .jpg, sedangkan subfolder *gt* (singkatan dari *ground truth*) berisi file dengan format .txt yang berfungsi sebagai anotasi. File anotasi tersebut memuat informasi posisi teks dalam gambar dalam bentuk koordinat, serta label teks yang sesuai. Pada folder *train*, terdapat sebanyak 1.000 gambar beserta 1.000 file anotasi. Semua citra dalam folder ini memiliki resolusi tetap sebesar 1280 × 720 piksel. Adapun folder *test* terdiri atas 500 gambar dan 500 file anotasi yang sesuai. Dataset ini merupakan standar internasional yang banyak digunakan dalam pengujian model deteksi teks, karena mencakup berbagai tantangan visual seperti teks buram, teks melengkung, dan latar belakang kompleks.

3.2. Arsitektur Sistem

Arsitektur *Faster* R-CNN merupakan pengembangan dari metode R-CNN klasik untuk deteksi objek, termasuk teks. Arsitektur ini terdiri atas beberapa komponen utama. Pertama, citra masukan diproses melalui jaringan konvolusional (CNN) untuk menghasilkan *feature map*. Kemudian, *Region Proposal Network (*RPN*)* mengambil peta fitur tersebut dan menghasilkan sejumlah area kandidat atau *proposals* yang berpotensi mengandung objek. Setiap proposal ini kemudian melewati proses *RoI (Region of Interest) Pooling*, yang bertujuan untuk menyesuaikan ukuran fitur agar seragam sebelum diklasifikasikan. Pada tahap

selanjutnya, dilakukan dua proses sekaligus, yaitu klasifikasi objek (melalui perhitungan *classification loss*) dan penyesuaian posisi *bounding box* (melalui perhitungan *bounding box regression loss*). Gambar ini menunjukkan alur deteksi objek yang efisien, di mana semua komponen saling terintegrasi dalam satu jaringan *end-to-end*. Berikut merupakan arsitektur *Faster* R-CNN ditunjukkan pada Gambar 2.1.



Gambar 2.1. Arsitektur *Faster* R-CNN [5]

3.3. *Region Proposal Network* (RPN)

Region Proposal Network (RPN) adalah sebuah jaringan saraf konvolusional yang dirancang khusus untuk menghasilkan kandidat area (*region proposals*) dalam suatu gambar, yang kemungkinan besar mengandung objek yang ingin dideteksi. RPN merupakan komponen inti dalam arsitektur *Faster* R-CNN, yang menggantikan metode pencarian wilayah tradisional seperti *Selective Search* yang digunakan dalam R-CNN klasik. Fungsi utama dari Region Proposal Network (RPN) adalah:

1. Menghasilkan wilayah proposal (*bounding box*) yang berpotensi mengandung objek dari gambar masukan.
2. Mengevaluasi skor kemungkinan objek (*objectness score*) untuk setiap wilayah, yaitu menilai apakah suatu area mengandung objek atau bukan.
3. Memberikan koordinat lokasi proposal (*bounding box coordinates*) untuk setiap area yang dianggap relevan.

Secara teknis, proses kerja RPN dapat dijelaskan dalam beberapa langkah seperti berikut ini:

1. *Input Feature Map* dari CNN *Backbone*
   a. Gambar masukan terlebih dahulu diproses oleh jaringan CNN (misalnya VGG-16, ResNet).
   b. Hasil dari CNN adalah *feature map*, yaitu representasi spasial dari fitur-fitur penting pada gambar.
2. *Sliding Window* di atas *Feature Map*
   a. RPN mengaplikasikan jendela konvolusional kecil (biasanya 3×3) yang meluncur (*sliding*) di atas feature map.
   b. Untuk setiap posisi jendela, RPN akan memproses sekelompok anchor box.
3. *Anchor Boxes*
   a. Pada setiap titik *grid*, dihasilkan beberapa *anchor box* dengan berbagai skala (ukuran) dan rasio aspek (lebar:tinggi).
   b. Contoh: 3 skala × 3 rasio = 9 *anchor box* per lokasi.
   c. Anchor adalah calon area bounding box yang akan dievaluasi.
4. *Output* dari RPN

   RPN memberikan dua keluaran utama untuk setiap *anchor*:
   a. *Objectness Score* merupakan probabilitas apakah *anchor* tersebut mengandung objek.
   b. *Bounding Box Regression* koreksi (*offset*) terhadap posisi *anchor* agar lebih presisi sesuai dengan objek asli.
5. *Non-Maximum Suppression* (NMS)

   Setelah semua *anchor* diberi skor dan lokasi, RPN akan menggunakan algoritma *Non-Maximum Suppression* (NMS) untuk menghapus wilayah yang tumpang tindih dan hanya menyimpan proposal terbaik.

3.4. Proses Pelatihan Model dan Evaluasi

Pelatihan model dilakukan dengan menggunakan *framework* PyTorch serta arsitektur deteksi objek *Faster* R-CNN dengan *backbone* ResNet-50. Pelatihan ini dilakukan untuk menyesuaikan model yang telah dilatih sebelumnya pada dataset

COCO agar mampu mengenali objek teks pada dataset ICDAR 2015. Berikut adalah tahapan pelatihan model secara sistematis:

1. Inisialisasi Model

   Model *Faster* R-CNN yang telah dilatih sebelumnya dimuat sebagai model dasar (*pretrained*). Bagian kepala klasifikator kemudian dikustomisasi agar sesuai dengan jumlah kelas yang dibutuhkan, yaitu dua kelas: latar belakang (*background*) dan teks.

2. Pemindahan Model ke GPU

   Seluruh parameter model dipindahkan ke perangkat GPU apabila tersedia. Hal ini bertujuan untuk mempercepat proses komputasi selama pelatihan.

3. Konfigurasi *Optimizer* dan *Scheduler*

   Untuk mengoptimalkan bobot model selama pelatihan, digunakan metode optimasi *Stochastic Gradient Descent* (SGD) dengan parameter seperti laju pembelajaran (*learning rate*), momentum, dan *weight decay*. *Scheduler* juga digunakan untuk menurunkan nilai learning rate secara bertahap pada setiap beberapa epoch guna menjaga stabilitas pelatihan.

4. Pemanggilan Dataset dan *Dataloader*

   Dataset ICDAR 2015 yang telah dikonversi ke format COCO dimuat menggunakan pustaka khusus yang menghubungkan gambar dan anotasinya. Citra dikonversi menjadi tensor, sedangkan data bounding box dan label teks disesuaikan agar dapat digunakan oleh model. Dataset dimuat dalam batch untuk efisiensi pelatihan.

5. Transformasi dan Pra-pemrosesan Citra

   Sebelum citra digunakan dalam pelatihan, dilakukan transformasi sederhana seperti konversi gambar ke bentuk tensor dan normalisasi nilai piksel. Hal ini dilakukan untuk menyesuaikan format masukan dengan kebutuhan model konvolusional.

6. Pelatihan Model per *Epoch*

   Pada setiap epoch, citra dan anotasinya diproses dalam beberapa *batch*. Model akan melakukan prediksi awal, menghitung total *loss* berdasarkan perbedaan antara prediksi dan anotasi, kemudian melakukan pembaruan

bobot menggunakan metode propagasi balik (*backpropagation*). Nilai *loss* total terdiri atas *classification loss*, *bounding box regression loss*, dan *objectness loss*.

7. Penyesuaian *Learning Rate*

   Setelah setiap epoch selesai, scheduler akan menurunkan nilai *learning rate* untuk menstabilkan proses pembelajaran dan mencegah pembelajaran yang terlalu cepat (*overshooting*).

8. Penyimpanan Model

   Setelah menyelesaikan pelatihan pada setiap *epoch*, bobot model disimpan ke dalam file agar dapat digunakan kembali untuk pengujian, pelatihan lanjutan, atau penerapan sistem pada aplikasi lain.

9. Pengulangan Siklus *Epoch*

   Proses pelatihan diulangi selama lima *epoch*. Jumlah *epoch* ini dipilih berdasarkan pertimbangan efisiensi waktu pelatihan serta batasan sumber daya komputasi yang tersedia.

   Rumus-rumus utama yang digunakan pada script:

   a. *Intersection over Union* (IoU) – dasar evaluasi

   $$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union} \quad (3.1)$$

   Area of Overlap = Luas area persegi panjang (*bounding box*) yang saling bertumpuk antara ground truth dan prediksi.

   Area of Union = Luas area gabungan dari *bounding box ground truth* dan prediksi (tidak *double count* bagian overlap).

   b. Precision

   $$Precision = \frac{TP}{TP+FP} \quad (3.2)$$

   TP (*True Positives*) = jumlah prediksi yang benar (IoU ≥ threshold)

   FP (*False Positives*) = jumlah prediksi salah (tidak cocok atau duplikat prediksi)

   c. Recall

   $$Recall = \frac{TP}{TP+FN} \quad (3.3)$$

   FN (*False Negatives*) = objek sebenarnya yang tidak terdeteksi oleh model

d. *Average Precision* (AP) – *area under PR curve*

$$AP = \frac{1}{101}\sum_{r\epsilon\{0.0,0.01,\ldots,1.0\}} Pinterp(r) \quad (3.4)$$

r = titik-titik recall dari 0.0 sampai 1.0 (sebanyak 101 titik)

$Pinterp(r)$ = nilai precision tertinggi untuk recall ≥ r (interpolasi)

e. *mean Average Precision* (mAP)

$$mAP = \frac{1}{10}\sum_{r\epsilon\{0.50,0.55,\ldots,0.95\}} AP@t \quad (3.5)$$

t = threshold IoU yang digunakan (dari 0.50 sampai 0.95 setiap 0.05)

$AP@t$ = nilai Average Precision pada setiap threshold t

f. *Average Recall* (AR)

$$AR = \frac{1}{N}\sum_{i=1}^{N} Recall_i \quad (3.6)$$

N = jumlah IoU threshold yang diuji

$Recall_i$ = nilai recall pada threshold ke-i

$maxDets$ = jumlah maksimum prediksi yang dinilai per gambar (misal 1, 10, 100)

BAB IV
HASIL DAN PEMBAHASAN

Bab ini membahas hasil implementasi serta analisis dari metode yang digunakan untuk mendeteksi teks dalam gambar menggunakan pendekatan berbasis *Faster Region-based Convolutional Neural Network* (R-CNN) untuk mendeteksi teks dalam *scene images*.

4.1. Konversi Anotasi Teks

Program ini digunakan untuk mengonversi dataset teks beranotasi dalam format ICDAR ke dalam format COCO JSON. Setiap gambar dalam dataset ICDAR dilengkapi dengan file label berformat ".txt" yang berisi koordinat poligon. Program ini membaca folder gambar beserta folder "ground-truth" ICDAR, kemudian secara otomatis memproses ukuran setiap gambar dan mengekstrak delapan titik poligon dari file label. Selanjutnya, program menghitung *bounding box* dan luas area poligon, lalu menyusun struktur JSON COCO yang mencakup daftar gambar, anotasi, dan kategori. Hasil konversi berupa file JSON yang kompatibel dengan berbagai *framework* deteksi objek atau teks, seperti Detectron2 dan MMDetection, sehingga memudahkan proses pelatihan atau evaluasi model tanpa perlu menulis parser manual untuk format ICDAR. Berikut merupakan program dari konversi *image* ke file COCO JSON.

```
import torch
import torchvision
from torch.utils.data import DataLoader
from torchvision.models.detection import FasterRCNN
from torchvision.models.detection.faster_rcnn import
FastRCNNPredictor
from torchvision.datasets import CocoDetection
from torchvision.transforms import functional as F
import matplotlib.pyplot as plt
from PIL import Image
```

```
# Load Faster R-CNN dengan backbone ResNet-50
def get_model(num_classes):
    # Load model pre-trained
    model =
torchvision.models.detection.fasterrcnn_resnet50_fpn(weights="DEFA
ULT")
    # Ambil jumlah input fitur dari classifier
    in_features =
model.roi_heads.box_predictor.cls_score.in_features
    # Ganti head classifier dengan yang baru sesuai jumlah kelas
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
num_classes)
    return model

# Inisialisasi
num_classes = 2  # 1 background + 1 kelas "Text"
device = torch.device('cuda') if torch.cuda.is_available() else
torch.device('cpu')

# Load model dan checkpoint
model = get_model(num_classes)
model.load_state_dict(torch.load(
    r"E:\Semester 6\AMV\fasterrcnn_resnet50_epoch_5.pth",
    map_location=device  # ← ini penting agar bisa jalan di CPU
))
model.to(device)
model.eval()

# Fungsi untuk mempersiapkan gambar
def prepare_image(image_path):
    image = Image.open(image_path).convert("RGB")
    image_tensor = F.to_tensor(image).unsqueeze(0)  # Tambah batch
dimensi
    return image_tensor.to(device)
```

```
# Prediksi
image_path = r"E:\Semester 6\AMV\coba1.jpg"
image_tensor = prepare_image(image_path)

with torch.no_grad():
    prediction = model(image_tensor)

# Label kelas (ubah sesuai kebutuhan)
COCO_CLASSES = {0: "Background", 1: "Text"}

def get_class_name(class_id):
    return COCO_CLASSES.get(class_id, "Unknown")

# Fungsi untuk menampilkan bounding box
def draw_boxes(image, prediction, fig_size=(12, 10)):
    boxes = prediction[0]['boxes'].cpu().numpy()
    labels = prediction[0]['labels'].cpu().numpy()
    scores = prediction[0]['scores'].cpu().numpy()

    plt.figure(figsize=fig_size)
    plt.imshow(image)

    threshold = 0.5
    for box, label, score in zip(boxes, labels, scores):
        if score > threshold:
            x_min, y_min, x_max, y_max = box
            class_name = get_class_name(label)
            plt.gca().add_patch(plt.Rectangle((x_min, y_min),
x_max - x_min, y_max - y_min,
                                                linewidth=2,
edgecolor='r', facecolor='none'))
            plt.text(x_min, y_min, f"{class_name} ({score:.2f})",
color='r', fontsize=10,
                     bbox=dict(facecolor='yellow', alpha=0.5))
```

```
    plt.axis('off')
    plt.show()

# Tampilkan hasil deteksi
draw_boxes(Image.open(image_path), prediction)
```

4.2. Library

Program ini merupakan bagian dari implementasi model deteksi objek berbasis Faster R-CNN menggunakan "framework PyTorch" dan" Torchvision". Program tersebut mengimpor sejumlah modul utama, antara lain "torch" dan "torchvision" untuk komputasi tensor dan model pra-terlatih, "DataLoader" untuk manajemen pengambilan data dalam batch, serta "FasterRCNN" untuk arsitektur inti deteksi objek. Selain itu, "FastRCNNPredictor" digunakan untuk mengadaptasi jumlah kelas keluaran sesuai kebutuhan. Dataset yang dipakai mengikuti format COCO, yang diakses melalui modul "CocoDetection". Keperluan visualisasi dan pra-pemrosesan gambar, program memanfaatkan "matplotlib.pyplot", "PIL.Image", dan fungsi transformasi dari "torchvision.transforms.functional (F)". Secara keseluruhan, kode ini menyiapkan komponen esensial untuk membangun dan menjalankan sistem deteksi objek dengan arsitektur Faster R-CNN pada dataset COCO. Berikut ini merupakan program untuk pemanggilan *library* yang digunakan pada metode Faster R-CNN.

```
import torch
import torchvision
from torch.utils.data import DataLoader
from torchvision.models.detection import FasterRCNN
from torchvision.models.detection.faster_rcnn import
FastRCNNPredictor
from torchvision.datasets import CocoDetection
from torchvision.transforms import functional as F
import matplotlib.pyplot as plt
from PIL import Image
```

4.3. Transformations

Program CocoTransform menerapkan transformasi untuk memproses dataset COCO dengan mengonversi gambar dari format PIL ke tensor menggunakan "F.to_tensor()". Anotasi target tetap dipertahankan dalam bentuk aslinya tanpa modifikasi untuk mendukung pelatihan model deteksi objek. Transformasi ini memastikan kesesuaian format *input* dengan arsitektur model *deep learning* sekaligus menjaga integritas data anotasi. Berikut program yang digunakan ditunjukkan di bawah ini.

```
# Define transformations
class CocoTransform:
    def __call__(self, image, target):
        image = F.to_tensor(image)  # Convert PIL image to tensor
        return image, target
```

4.4. Load Dataset

Program ini digunakan untuk memuat dataset format COCO (contohnya ICDAR 2015 hasil konversi) guna keperluan pelatihan model deteksi objek berbasis PyTorch. Fungsi "get_coco_dataset" membuat *instance* "CocoDetection" dengan transformasi gambar ke tensor, sementara "train_loader" dan "val_loader" mengatur pengambilan data dalam *batch* selama proses pelatihan dan validasi. Melalui "DataLoader" sistem menjamin pengacakan data saat pelatihan dan pengurutan yang konsisten saat validasi, dengan dukungan penanganan format khusus seperti *tuple* (*image*, target) melalui fungsi "collate_fn". Berikut ini merupakan program yang digunakan untuk *load* dataset.

```
# Dataset class
def get_coco_dataset(img_dir, ann_file):
    return CocoDetection(
        root=img_dir,
        annFile=ann_file,
        transforms=CocoTransform()
    )

# Load datasets
```

```
train_dataset = get_coco_dataset(
    img_dir="/content/drive/MyDrive/icdar15/train/images",
    ann_file="/content/drive/MyDrive/icdar15/instances_train.json"
)


val_dataset = get_coco_dataset(
    img_dir="/content/drive/MyDrive/icdar15/test/images",
    ann_file="/content/drive/MyDrive/icdar15/instances_test.json"
)



# DataLoader
train_loader = DataLoader(train_dataset, batch_size=4,
shuffle=True, collate_fn=lambda x: tuple(zip(*x)))
val_loader = DataLoader(val_dataset, batch_size=4, shuffle=False,
collate_fn=lambda x: tuple(zip(*x)))
```

Adapun hasil dari *run* program *load* dataset di atas yang dapat di tunjukkan pada Gambar 4.1.

```
...  loading annotations into memory...
     Done (t=0.76s)
     creating index...
     index created!
     loading annotations into memory...
     Done (t=0.86s)
     creating index...
     index created!
```

Gambar 4.1. Hasil *Run Load* Dataset

4.5. Model *Faster* R-CNN

Program ini merupakan fungsi untuk "get_model" berperan dalam inisialisasi model Faster R-CNN menggunakan arsitektur ResNet-50 sebagai *backbone* dengan bobot pre-trained. Parameter "num_classes" berfungsi untuk menentukan jumlah kategori objek yang akan dikenali. Proses eksekusi diawali dengan pemanggilan model dasar melalui

"torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)", dilanjutkan dengan ekstraksi dimensi fitur *input* pada layer klasifikasi. Tahap akhir melibatkan penggantian layer prediksi bawaan dengan "FastRCNNPredictor" yang telah dikonfigurasi sesuai kebutuhan klasifikasi spesifik. *Output* fungsi berupa model yang telah terkonfigurasi untuk proses pelatihan maupun implementasi pada dataset target. Berikut merupakan program model Faster R-CNN di bawah ini.

```
# Load Faster R-CNN with ResNet-50 backbone
def get_model(num_classes):
    # Load pre-trained Faster R-CNN
    model =
torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=Tr
ue)

    # Get the number of input features for the classifier
    in_features =
model.roi_heads.box_predictor.cls_score.in_features

    # Replace the pre-trained head with a new one
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
num_classes)
    return model
# Initialize the model
num_classes = 2
model = get_model(num_classes)
```

4.6. Konfigurasi Perangkat, Optimizer, dan Scheduler

Program ini melakukan konfigurasi awal model deteksi objek untuk optimasi proses pelatihan. Model dipindahkan ke perangkat komputasi (GPU/CPU) melalui metode "model.to(device)", diikuti dengan inisialisasi optimizer SGD (*Stochastic Gradient Descent*) yang mengatur pembaruan parameter dengan *learning rate* 0.005, momentum 0.9, dan *weight decay* 0.0005. Meningkatkan konvergensi, diterapkan *learning rate scheduler* yang mengurangi *learning rate* secara eksponensial dengan faktor 0.1 setiap 3 *epoch*. Berikut merupakan program konfigurasi perangkat, *optimizer* dan *scheduler* di bawah ini.

```
# Move model to GPU if available
device = torch.device('cuda') if torch.cuda.is_available() else
torch.device('cpu')
model.to(device)

# Define optimizer and learning rate scheduler
params = [p for p in model.parameters() if p.requires_grad]
optimizer = torch.optim.SGD(params, lr=0.005, momentum=0.9,
weight_decay=0.0005)
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer,
step_size=3, gamma=0.1)
```

4.5. Train One Epoch

Program ini digunakan untuk fungsi "train_one_epoch" menjalankan proses pelatihan model deteksi objek selama satu siklus lengkap (*epoch*) pada dataset. Implementasinya meliputi beberapa tahapan utama (1) mengaktifkan mode pelatihan model, (2) memproses data dalam bentuk batch dengan transfer data ke perangkat komputasi (GPU/CPU), (3) melakukan validasi dan penyaringan *bounding box*, serta (4) menghitung nilai loss berdasarkan prediksi model. Proses optimasi dilakukan melalui "backpropagation" dengan memanfaatkan *optimizer* yang telah dikonfigurasi. Fungsi ini secara otomatis melewati *batch* yang tidak mengandung anotasi valid dan menampilkan akumulasi nilai loss di akhir *epoch* sebagai metrik evaluasi kinerja pelatihan. Berikut ini merupakan program untuk *train one epoch* di bawah ini.

```
def train_one_epoch(model, optimizer, data_loader, device, epoch):
    model.train()
    for images, targets in data_loader:
        # Move images to the device
        images = [img.to(device) for img in images]

        # Validate and process targets
        processed_targets = []
        valid_images = []
        for i, target in enumerate(targets):
```

```
            boxes = []
            labels = []
            for obj in target:
                # Extract bbox
                bbox = obj["bbox"]  # Format: [x, y, width,
height]
                x, y, w, h = bbox

                # Ensure the width and height are positive
                if w > 0 and h > 0:
                    boxes.append([x, y, x + w, y + h])  # Convert
to [x_min, y_min, x_max, y_max]
                    labels.append(obj["category_id"])
            # Only process if there are valid boxes
            if boxes:
                processed_target = {
                    "boxes": torch.tensor(boxes,
dtype=torch.float32).to(device),
                    "labels": torch.tensor(labels,
dtype=torch.int64).to(device),
                }
                processed_targets.append(processed_target)
                valid_images.append(images[i])  # Add only valid
images

        # Skip iteration if no valid targets
        if not processed_targets:
            continue
        # Ensure images and targets are aligned
        images = valid_images
        # Forward pass
        loss_dict = model(images, processed_targets)
        losses = sum(loss for loss in loss_dict.values())
        # Backpropagation
        optimizer.zero_grad()
        losses.backward()
```

```
        optimizer.step()

    print(f"Epoch [{epoch}] Loss: {losses.item():.4f}")
```

4.6. Training Loop

Proses pelatihan model Faster R-CNN diimplementasikan dengan menjalankan 5 siklus *epoch* secara berurutan. Pada setiap *epoch*, fungsi "train_one_epoch" melakukan optimasi parameter model menggunakan data *training*, diikuti dengan penyesuaian learning rate melalui "lr_scheduler.step()". Setelah menyelesaikan setiap *epoch*, *state* model beserta seluruh parameternya disimpan dalam format file ".pth" menggunakan "torch.save()", memungkinkan proses *loading* model untuk inferensi tanpa perlu *retraining*. Implementasi ini merepresentasikan alur standar pelatihan dan penyimpanan model *deep learning* dalam f*ramework* PyTorch. Berikut ini merupakan program *training loop* di bawah ini.

```
# Training loop
num_epochs = 5
for epoch in range(num_epochs):
    train_one_epoch(model, optimizer, train_loader, device, epoch)
    lr_scheduler.step()

    # Save the model's state dictionary after every epoch
    model_path = f"fasterrcnn_resnet50_epoch_{epoch + 1}.pth"
    torch.save(model.state_dict(), model_path)
    print(f"Model saved: {model_path}")
```

Adapun hasil *run* dari program *training loop* di atas yang dapat ditujukkan pada Gambar 4.2.

```
...  Epoch [0] Loss: 0.6919
     Model saved: fasterrcnn_resnet50_epoch_1.pth
     Epoch [1] Loss: 0.5588
     Model saved: fasterrcnn_resnet50_epoch_2.pth
     Epoch [2] Loss: 0.5110
     Model saved: fasterrcnn_resnet50_epoch_3.pth
     Epoch [3] Loss: 0.6795
     Model saved: fasterrcnn_resnet50_epoch_4.pth
     Epoch [4] Loss: 0.3696
     Model saved: fasterrcnn_resnet50_epoch_5.pth
```

Gambar 4.2. Hasil *Run Training Loop*

4.7. Testing

Program ini melakukan proses inferensi menggunakan model Faster R-CNN yang telah dilatih sebelumnya untuk mendeteksi objek pada gambar input. Implementasinya meliputi beberapa tahapan utama (1) memuat model dari file bobot ".pth" dan mengaktifkan mode evaluasi, (2) mengubah gambar input menjadi tensor, (3) melakukan prediksi untuk mendapatkan *bounding box*, label kelas, dan *confidence score*, serta (4) memvisualisasikan hasil deteksi dengan menampilkan kotak merah beserta informasi kelas dan skor kepercayaan menggunakan "Matplotlib". Fungsi ini memungkinkan evaluasi performa model pada data baru dilengkapi dengan visualisasi hasil deteksi. Berikut ini merupakan program testing di bawah ini.

```
import torch
import torchvision
from torch.utils.data import DataLoader
from torchvision.models.detection import FasterRCNN
from torchvision.models.detection.faster_rcnn import
FastRCNNPredictor
from torchvision.datasets import CocoDetection
from torchvision.transforms import functional as F
import matplotlib.pyplot as plt
from PIL import Image

# Load Faster R-CNN with ResNet-50 backbone
def get_model(num_classes):
    # Load pre-trained Faster R-CNN
    model =
torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=Tr
ue)
    # Get the number of input features for the classifier
    in_features =
model.roi_heads.box_predictor.cls_score.in_features
    # Replace the pre-trained head with a new one
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
num_classes)
    return model
```

```
# Initialize the model
num_classes = 2  # Background + chair + person + table

# Move model to GPU if available
device = torch.device('cuda') if torch.cuda.is_available() else
torch.device('cpu')


# Load the trained model
model = get_model(num_classes)
model.load_state_dict(torch.load("/content/drive/MyDrive/fasterrcn
n_resnet50_epoch_5.pth"))
model.to(device)
model.eval()  # Set the model to evaluation mode


def prepare_image(image_path):
    image = Image.open(image_path).convert("RGB")  # Open image
    image_tensor = F.to_tensor(image).unsqueeze(0)  # Convert
image to tensor and add batch dimension
    return image_tensor.to(device)



# Load the unseen image
image_path = "/content/drive/MyDrive/coba.jpg"
image_tensor = prepare_image(image_path)

with torch.no_grad():  # Disable gradient computation for
inference
    prediction = model(image_tensor)

# `prediction` contains:
# - boxes: predicted bounding boxes
# - labels: predicted class labels
# - scores: predicted scores for each box (confidence level)
```

```
COCO_CLASSES = {0: "Background", 1: "Text"}

def get_class_name(class_id):
    return COCO_CLASSES.get(class_id, "Unknown")

# Draw bounding boxes with the correct class names and increase
image size
def draw_boxes(image, prediction, fig_size=(10, 10)):
    boxes = prediction[0]['boxes'].cpu().numpy()  # Get predicted
bounding boxes
    labels = prediction[0]['labels'].cpu().numpy()  # Get
predicted labels
    scores = prediction[0]['scores'].cpu().numpy()  # Get
predicted scores

    # Set a threshold for showing boxes (e.g., score > 0.5)
    threshold = 0.5

    # Set up the figure size to control the image size
    plt.figure(figsize=fig_size)  # Adjust the figure size here

    for box, label, score in zip(boxes, labels, scores):
        if score > threshold:
            x_min, y_min, x_max, y_max = box
            class_name = get_class_name(label)  # Get the class
name
            plt.imshow(image)  # Display the image
            plt.gca().add_patch(plt.Rectangle((x_min, y_min),
x_max - x_min, y_max - y_min,
                                                linewidth=2,
edgecolor='r', facecolor='none'))
            plt.text(x_min, y_min, f"{class_name} ({score:.2f})",
color='r')

    plt.axis('off')  # Turn off axis
```

```
    plt.show()

# Display the image with bounding boxes and correct labels
draw_boxes(Image.open(image_path), prediction, fig_size=(12, 10))
```

Adapun hasil *run* program *testing* dari program di atas yang dapat ditunjukkan pada Gambar 4.3.

Gambar 4.3. Hasil Testing menggunakan Metode R-CNN

4.8. Evaluasi

Bagian ini menjelaskan proses evaluasi model deteksi objek untuk bintang laut menggunakan Mean Average Precision (mAP) sebagai metrik utama. Berikut ini merupakan program dari evaluasi di bawah ini.

```
# 1. Install & Import
!pip install torch torchvision pycocotools --quiet

import torch
import torchvision
```

```python
from torch.utils.data import DataLoader
from torchvision.models.detection.faster_rcnn import
FastRCNNPredictor
from torchvision.datasets import CocoDetection
from torchvision.transforms import functional as F
from PIL import Image
import os
import json
from tqdm import tqdm
from pycocotools.coco import COCO
from pycocotools.cocoeval import COCOeval

# 2. Patch GT COCO JSON jika tidak ada 'info'
def ensure_info_field(json_path, out_path=None):
    with open(json_path, "r") as f:
        coco_data = json.load(f)
    if "info" not in coco_data:
        coco_data["info"] = {}
    if out_path is None:
        out_path = json_path
    with open(out_path, "w") as f:
        json.dump(coco_data, f)
    return out_path

# 3. Path dataset dan model
img_root = "/content/drive/MyDrive/icdar15/test/images"
ann_file = "/content/drive/MyDrive/icdar15/instances_test.json"
model_path =
"/content/drive/MyDrive/icdar15/fasterrcnn_resnet50_epoch_5.pth"

# Patch ann_file jika perlu
ann_file_fixed = ensure_info_field(ann_file,
"ann_file_fixed.json")

# 4. Load Model
```

```
num_classes = 2  # background + text
device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")

def get_model(num_classes):
    model =
torchvision.models.detection.fasterrcnn_resnet50_fpn(weights=None)
    in_features =
model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
num_classes)
    return model

model = get_model(num_classes)
model.load_state_dict(torch.load(model_path, map_location=device))
model.to(device)
model.eval()

# 5. Dataset & DataLoader
class CocoTest(torch.utils.data.Dataset):
    def _init_(self, img_folder, ann_file, transform=None):
        self.coco = COCO(ann_file)
        self.ids = list(sorted(self.coco.imgs.keys()))
        self.img_folder = img_folder
        self.transform = transform
    def _getitem_(self, idx):
        img_id = self.ids[idx]
        img_info = self.coco.loadImgs(img_id)[0]
        img_path = os.path.join(self.img_folder,
img_info['file_name'])
        img = Image.open(img_path).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, img_id, img.size
    def _len_(self):
```

```
        return len(self.ids)

dataset = CocoTest(img_root, ann_file_fixed,
transform=F.to_tensor)
dataloader = DataLoader(dataset, batch_size=1, shuffle=False,
num_workers=2, collate_fn=lambda x: x)

# 6. Inference & COCO-format result
results = []
for batch in tqdm(dataloader):
    img, img_id, img_size = batch[0]
    img = img.unsqueeze(0).to(device)
    with torch.no_grad():
        output = model(img)[0]
    boxes = output['boxes'].cpu().numpy()
    scores = output['scores'].cpu().numpy()
    labels = output['labels'].cpu().numpy()
    # Threshold (misal 0.5)
    for box, score, label in zip(boxes, scores, labels):
        if score < 0.5:
            continue
        x1, y1, x2, y2 = box
        width = x2 - x1
        height = y2 - y1
        results.append({
            "image_id": int(img_id),
            "category_id": int(label),
            "bbox": [float(x1), float(y1), float(width),
float(height)],
            "score": float(score)
        })

# 7. Save result json
with open("predictions.json", "w") as f:
    json.dump(results, f)
```

```
# 8. COCO Evaluation
coco_gt = COCO(ann_file_fixed)
coco_dt = coco_gt.loadRes("predictions.json")
cocoEval = COCOeval(coco_gt, coco_dt, "bbox")
cocoEval.evaluate()
cocoEval.accumulate()
cocoEval.summarize()
```

Adapun hasil dari *run* program evaluasi yang dapat ditunjukkan pada Gambar 4.4.

```
loading annotations into memory...
Done (t=0.03s)
creating index...
index created!
100%|██████████| 500/500 [01:10<00:00,  7.07it/s]
loading annotations into memory...
Done (t=0.02s)
creating index...
index created!
Loading and preparing results...
DONE (t=0.03s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=2.07s).
Accumulating evaluation results...
DONE (t=0.10s).
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.298
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.561
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.280
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.185
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.373
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.353
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.061
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.312
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.400
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.304
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.472
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.435
```

Gambar 4.4. Hasil Evaluasi

BAB V
KESIMPULAN

Berdasarkan penelitian yang telah dilakukan, dapat disimpulkan bahwa metode *Faster* R-CNN berhasil diimplementasikan untuk mendeteksi teks dalam gambar lingkungan nyata (*scene images*). Model dilatih menggunakan dataset ICDAR 2015 yang telah dikonversi ke format COCO, dengan pendekatan transfer learning dari model yang telah dilatih sebelumnya pada dataset COCO. Pelatihan dilakukan selama lima *epoch* menggunakan *framework* PyTorch dan arsitektur *backbone* ResNet-50. Model ini mampu mengenali fitur visual kompleks dalam gambar dan menghasilkan deteksi terhadap teks secara cukup akurat. Berdasarkan hasil evaluasi menggunakan metrik COCO, diperoleh nilai *Mean Average Precision* (mAP) sebesar 0,298 pada rentang IoU 0,50 hingga 0,95 dan nilai *average recall* sebesar 0,400, yang menandakan kinerja model berada pada tingkat sedang. Deteksi terhadap objek berukuran sedang dan besar relatif lebih baik, sedangkan performa untuk objek kecil masih cukup rendah.

Hasil ini menunjukkan bahwa Faster R-CNN merupakan pendekatan yang menjanjikan untuk deteksi teks dalam citra nyata, meskipun masih terdapat ruang untuk peningkatan kinerja. Perbaikan dapat dilakukan melalui penambahan jumlah *epoch*, penyesuaian ukuran *anchor box*, penerapan teknik augmentasi data untuk memperluas variasi masukan, serta *tuning* hyperparameter agar proses pelatihan menjadi lebih optimal. Peningkatan resolusi dan *pre-processing* citra dapat mendukung keberhasilan model dalam mengenali teks berukuran kecil atau dengan kontras rendah. Dengan pengembangan lebih lanjut, sistem ini memiliki potensi untuk diadaptasi pada berbagai aplikasi pengenalan teks berbasis citra, seperti sistem navigasi visual, pembacaan plat nomor otomatis, dan pembantu aksesibilitas visual.

DAFTAR PUSTAKA

[1] Basthomi, M. S. Y., & Utaminingrum, F. (2021). Deteksi dan pengenalan plat nama ruangan menggunakan Faster-RCNN dan PyTesseract pada purwarupa kursi roda pintar. Prosiding Seminar Nasional Teknologi Informasi dan Komunikasi (SENTIKA), Universitas Islam Indonesia.

[2] Darmanto, B. F., Setianingsih, C., & Saputra, R. E. (2023). Deteksi pelanggaran pada bahu jalan tol menggunakan algoritma Cascade R-CNN. Jurnal Teknologi Informasi dan Komputer, Vol. 10 No.2: 85–91.

[3] Chaitra, Y. L., Roopa, M. J., Gopalakrishna, M. T., Swetha, M. D., & Aditya, C. R. (2023). Text detection and recognition from scene images using Faster R-CNN and EasyOCR. International Journal of Computer Applications, Vol. 182 No. 45: 1–6.

[4] Cahyadi, N., Salsabila, S., Raniprima, S., & Monita, V. (2024). Tinjauan literatur teknik pengenalan plat nomor berbasis deep learning. Jurnal Teknologi dan Sistem Komputer, Vol. 12 No.1: 33–40.

[5] Popović, M. (2023). R-CNN, Fast R-CNN, and Faster R-CNN explained with a demonstration in PyTorch. DataHacker. Diakses pada 26 Juni 2025 dari: https://datahacker.rs/027-r-cnn-fast-r-cnn-and-faster-r-cnn-explained-with-a-demonstration-in-pytorch/